DOCENTE	Paolo BELLONI [paolo.belloni@its-ictpiemonte.it]
Unità Formativa	**Sistemi di Virtualizzazione**
Argomento	Infrastruttura Virtuale

Intervento realizzato da

Infrastruttura Virtuale

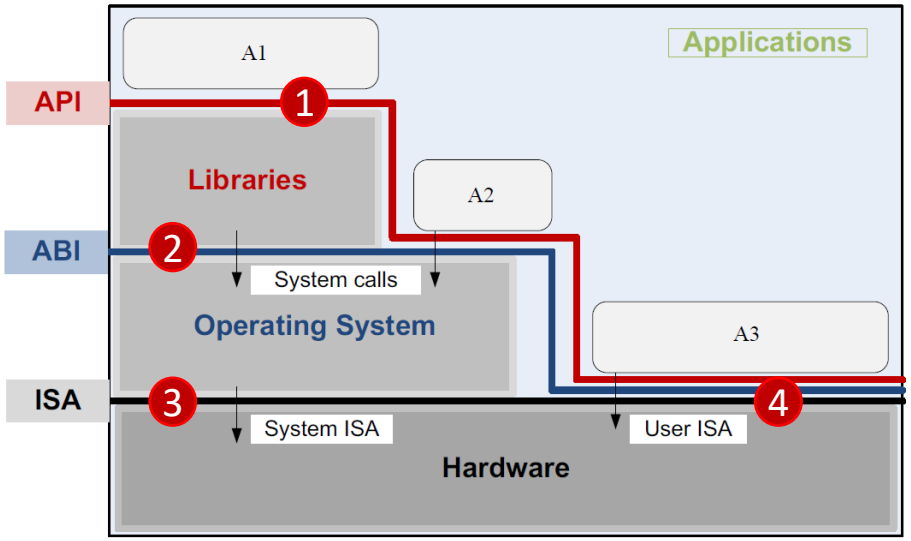
Componenti software (softwarizzate, astratte) che abilitano l'esecuzione di applicazioni senza disporre di una macchina fisica

- **Macchine Virtuali**
- **Container**
- **K8S**
- **Cloud Service Provider**

I livelli di Virtualizzazione 1/4

A che livello eseguire la virtualizzazione?

- Dipende dalle interfacce offerte dai vari componenti del sistema.
  - 4 ▪ Interfaccia tra HW e SW (user ISA: istruzioni macchina non privilegiate, invocabili da ogni programma).
  - 3 ▪ Interfaccia tra HW e SW (system ISA: istruzioni macchina invocabili solo da programmi privilegiati).
  - 2 ▪ Chiamate di Sistema: Application Binary Interface.
  - 1 ▪ Chiamate di libreria: Application Programming Interface.
- Obiettivo della virtualizzazione
  - ▪ Imitare il comportamento di queste interfacce



I livelli di Virtualizzazione 2/4

Cinque livelli di virtualizzazione

- Livello ISA (Instruction Set Architecture).
  - Si implementa emulando l'ISA, attraverso una traduzione di ogni singola istruzione oppure attraverso *dynamic binary translation* (converte in blocchi piuttosto che istruzione per istruzione).
- Livello Hardware (aka HW Abstraction Layer HAL oppure system VM) .
  - Bastato su hypervisor.
- Livello Sistema Operativo (aka containers).
- Livello Run-time Library.
- Livello applicazione utente (aka process VM).

I livelli di Virtualizzazione 3/4

Level of Virtualization	Functional Description	Example Packages	Merits, App Flexibility/ Isolation, Implementation Complexity
Instruction Set Architecture	Emulation of a guest ISA by host	Dynamo, Bird, Bochs, Crusoe	Low performance, high app flexibility, median complexity and isolation
Hardware-Level Virtualization	Virtualization on top of bare-metal hardware	XEN, VMWare, Virtual PC	High performance and complexity, median app flexibility, and good app isolation
Operating System Level	Isolated containers of user app with isolated resources	Docker Engine, Jail, FVM	Highest performance, low app flexibility and best isolation, and average complexity
Run-Time Library Level	Creating VM via run-time library through API hooks	Wine, vCUDA, WABI, LxRun	Average performance, low app flexibility and isolation, and low complexity
User Application Level	Deploy HLL VMs at user app level	JVM, .NET CLR, Panot	Low performance and app flexibility, very high complexity and app isolation

I livelli di Virtualizzazione 4/4

Level of Implementation	Higher Performance	Application Flexibility	Implementation Complexity	Application Isolation
Instruction Set Architecture (ISA)	X	XXXXX	XXX	XXX
Hardware-Level Virtualization	XXXXX	XXX	XXXXX	XXXX
Operating System Level	XXXXX	XX	XXX	XX
Run-Time Library Support	XXX	XX	XX	XX
User Application Level	XX	XX	XXXXX	XXXXX

VM – Virtualizzazione a livello HW

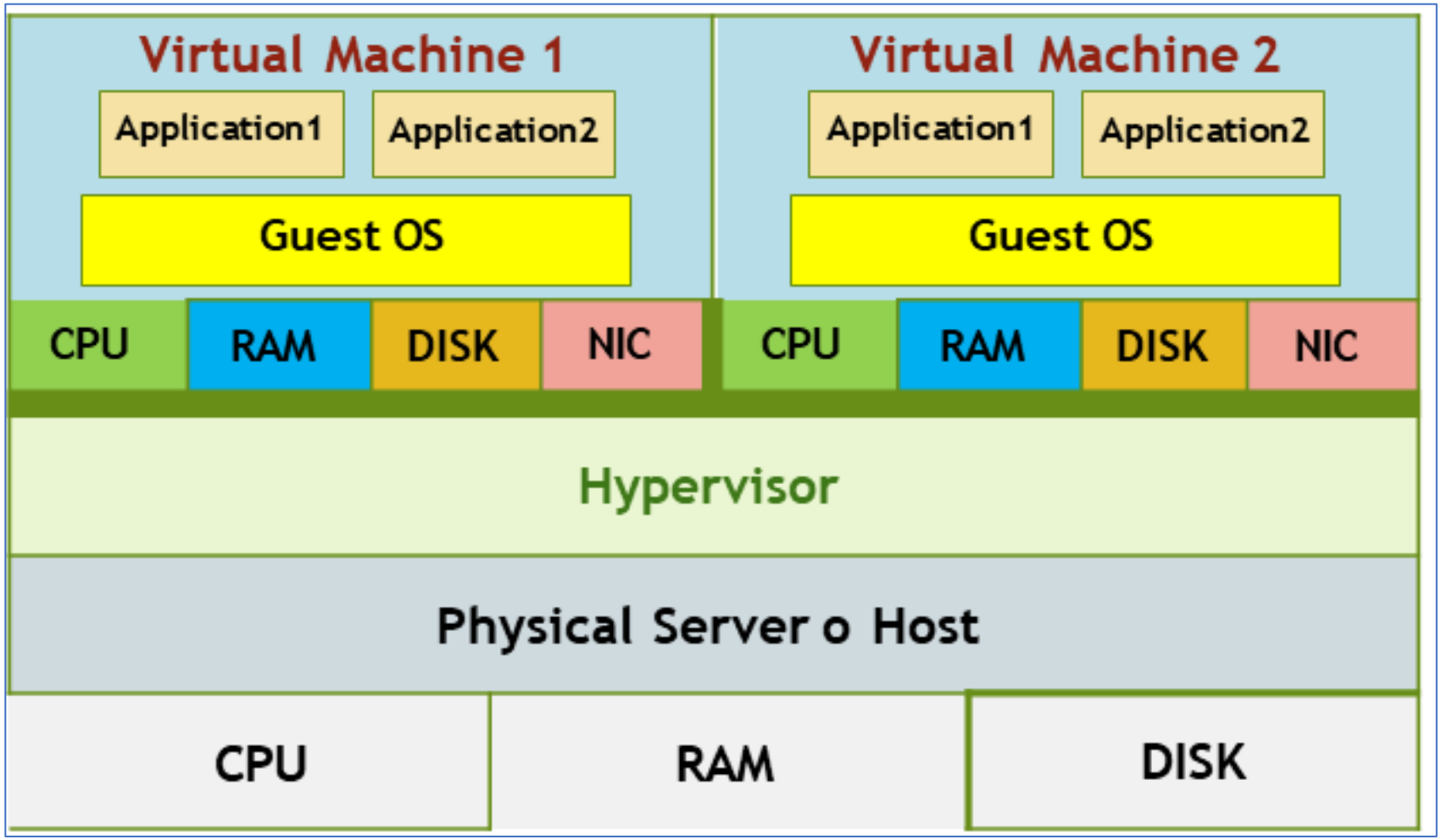
La Virtualizzazione a livello HW (System Level VM)

- La virtualizzazione a livello HW o system level VM si ottiene attraverso un hypervisor.

Definizioni

- Un **Hypervisor** è un software di virtualizzazione che permette di eseguire diversi SO contemporaneamente sulla stessa macchina fisica, interagendo direttamente con le risorse fisiche del server. L'hypervisor ha due componenti:
  - un **Kernel** – un SO per sistemi operativi;
  - un **Virtual Machine Monitor** (VMM), che astrae l'HW per apparire come una macchina fisica (con CPU, memoria e I/O propri). A volte VMM sinonimo di hypervisor.
- **Host** – piattaforma base sulla quale sono eseguite le varie VM.
  - Composta da macchina fisica, opzionalmente SO host, hypervisor.
- **Guest** – tutto quanto si trova all'interno di una VM.
  - SO guest e applicazioni eseguite dentro la VM.

La Virtualiz. a livello HW e l'Allocazione delle Risorse



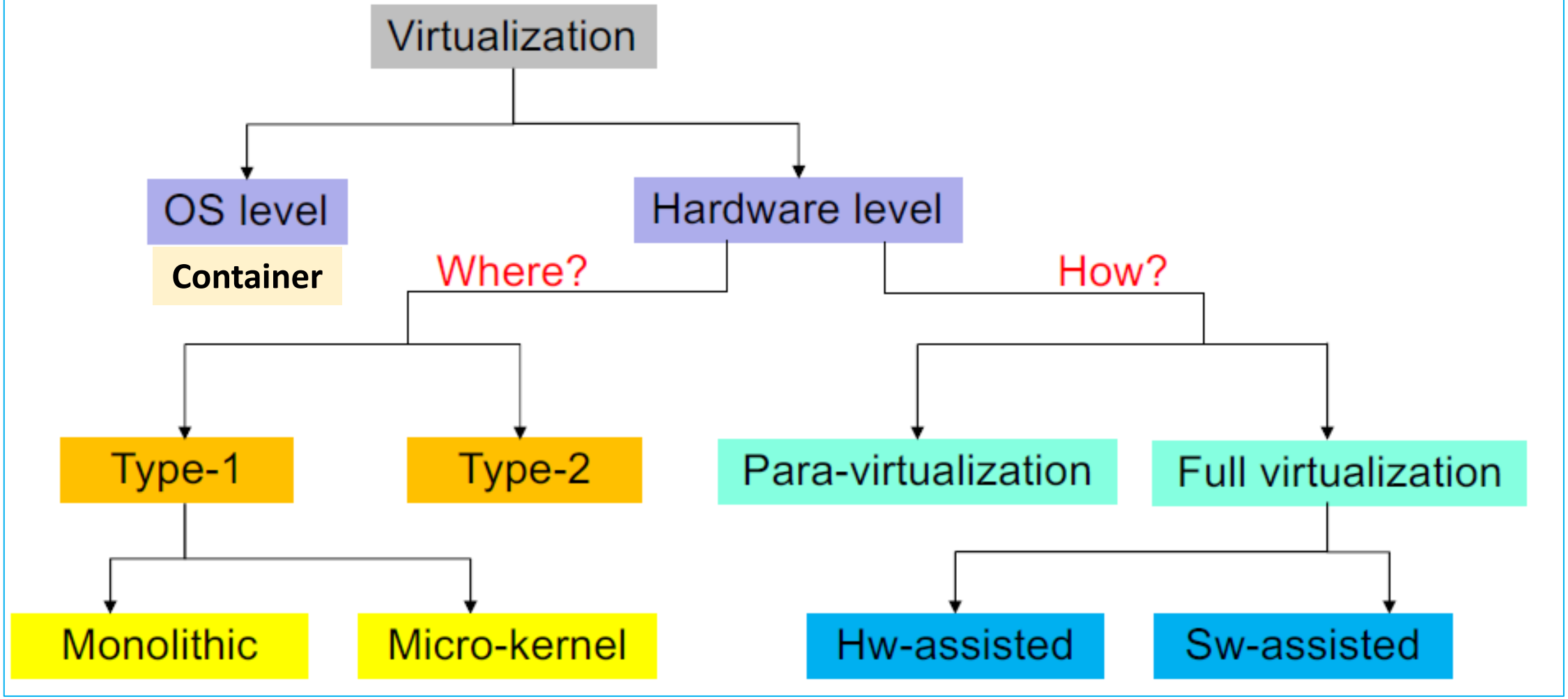
Classificazione della Virtualizzazione a livello HW

Esistono diversi tipi di virtualizzazione a livello HW (o system level virtualization), distinti in base a:

- Dove si istanza l'hypervisor:
  - System hypervisor, o di sistema, chiamato anche type-1, native or bare-metal hypervisor;
  - Hosted hypervisor, o ospitato, chiamato anche type-2 hypervisor.

- Come si virtualizzano le istruzioni sensibili e non virtualizzabili.
  - Full virtualization:
    - Software-assisted;
    - Hardware-assisted.
  - Para-virtualization.

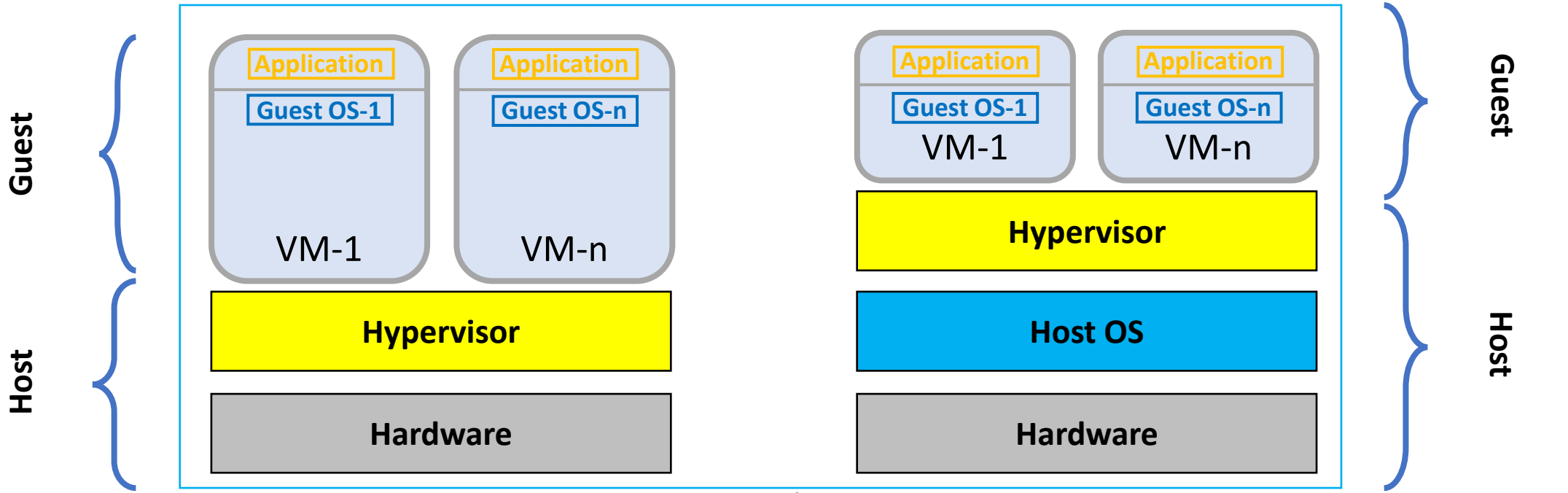
Classificazione della Virtualizzazione a livello HW



Hypervisor di Sistema vs Ospitato 1/2

In quale livello dell'architettura di sistema si colloca l'hypervisor?

- Direttamente sull'hardware: **hypervisor di sistema**
- Sopra il SO host: **hypervisor ospitato**



Hypervisor di sistema

Hypervisor ospitato

Hypervisor di Sistema vs Ospitato 2/2

- **Hypervisor di sistema (type-1)**: eseguito direttamente sull'hw, offre funzionalità di virtualizzazione integrate in un SO semplificato (è, di fatto, un SO).
  - L'hypervisor può avere un'architettura a micro-kernel (solo funzioni di base, no device driver) o monolitica.
  - Richiede HW certificato.
  - Esempi: Xen, KVM (non univocamente riconosciuto), VMware ESX, Hyper-V.
- **Hypervisor ospitato (type-2)**: eseguito sul SO host, accede alle risorse hw tramite le chiamate di sistema del SO host.
  - Interagisce con il SO host tramite l'ABI ed emula l'ISA di hw virtuale per i SO guest.
  - Vantaggio: può usare il SO host per gestire le periferiche ed utilizzare servizi di basso livello (es. scheduling delle risorse).
  - Vantaggio: non occorre modificare il SO guest.
  - Svantaggio: degrado delle prestazioni rispetto a hypervisor di sistema
  - Esempi: Bochs, Parallels Desktop, VirtualBox.

Full-Virtualization vs Para-Virtualization

Quale modalità di dialogo tra la VM e hypervisor per l'accesso alle risorse fisiche, ovvero come gestire l'esecuzione di istruzioni privilegiate?

- **Virtualizzazione completa**
- **Paravirtualizzazione**

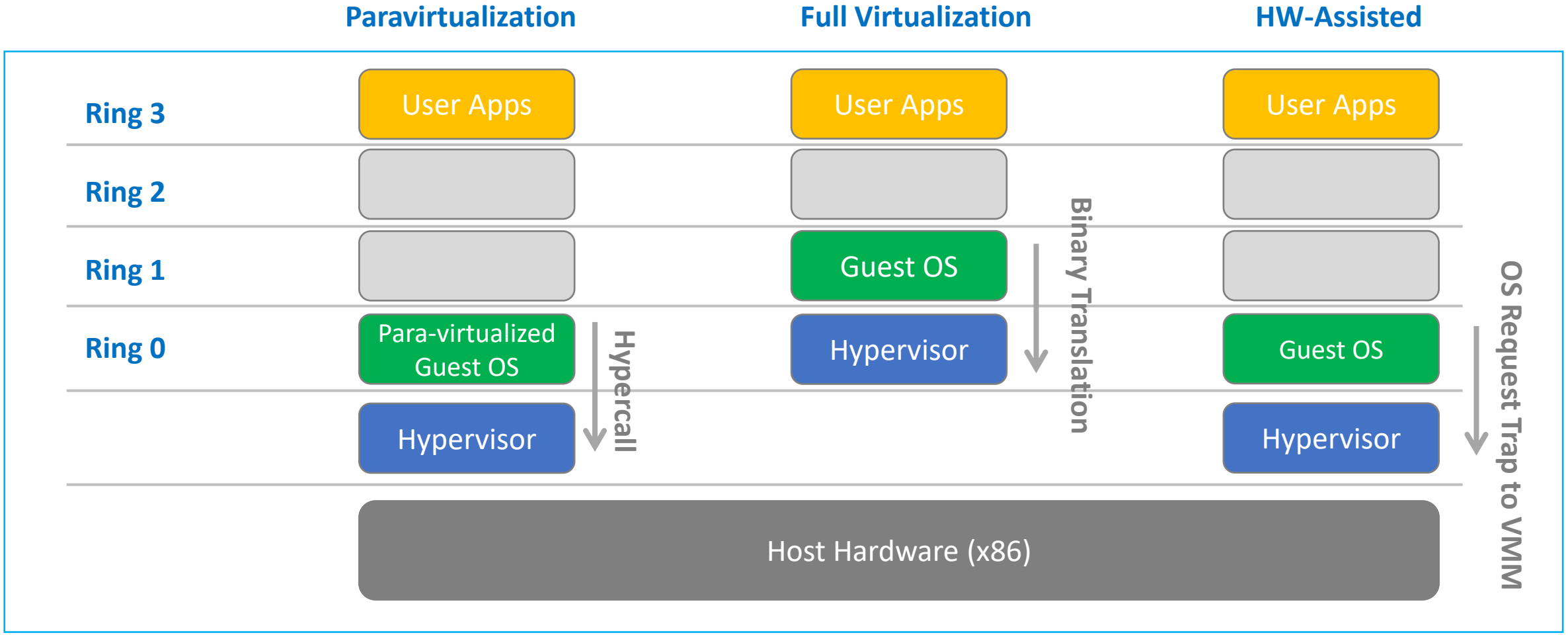
Virtualizzazione completa (full)

- L'hypervisor espone ad ogni VM interfacce hw simulate *funzionalmente identiche* a quelle della sottostante macchina fisica.
- L'hypervisor *intercetta* le richieste di accesso privilegiato all'hw (ad es. istruzioni di I/O) e ne emula il comportamento atteso.
- Esempi: KVM, VMware ESXi, Microsoft Hyper-V.

Para-Virtualizzazione

- L'hypervisor espone ad ogni VM interfacce hw simulate *funzionalmente simili* (ma non identiche) a quelle della sottostante macchina fisica.
- Non viene emulato l'hw, ma viene creato uno strato minimale di sw (Virtual Hardware API) per assicurare la gestione delle VM ed il loro isolamento.
- Esempi: Xen, Oracle VM (basato su Xen), PikeOS.

Riassumendo i diversi approcci di Virtualizzazione



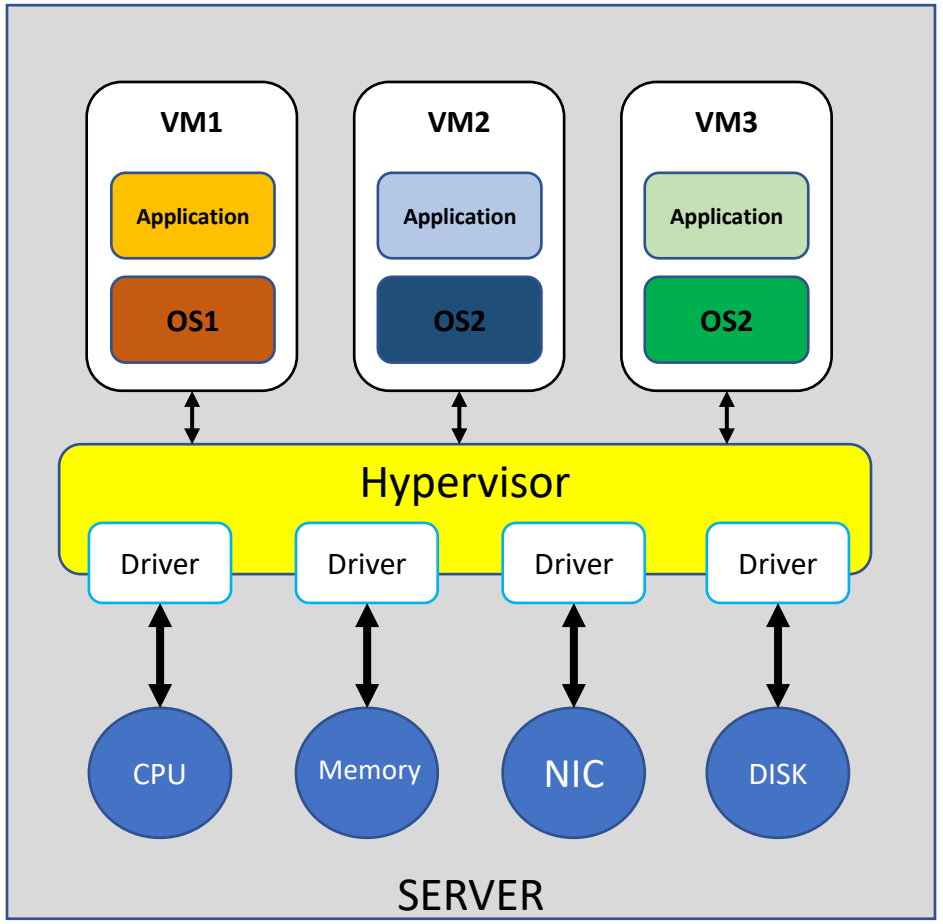
Xen, KVM

VMware ESX/ESXi, Microsoft Hyper-V

Intel VT-x, AMD-V

L'operatività della Virtual Machine (VM)

- Dal punto di vista dell'utente, una VM è un *logical compute system* (sistema logico per il processamento dei dati).
  - Lavora su un SO e ha applicazioni come una macchina fisica.
  - Dispone di componenti virtuali come CPU, RAM, dischi e interface di rete.
- Dal punto di vista dell'hypervisor invece:
  - Una VM è un insieme discreto di file (file di configurazione, file di dischi virtuali, file di virtual BIOS, file di VM swap e log file).



Source: **EMC²**, Cloud Infrastructure and Service, Student Guide, 2011

I file della Virtual Machine (VM) 1/2

File name	Description
Virtual BIOS File	• Stores the state of the virtual machine's (VM's) BIOS
Virtual Swap File	• Is a VM's paging file which backs up the VM RAM contents • The file exists only when VM is running
Virtual Disk File	• Stores the contents of the VM's disk drive • Appears like a physical disk drive to VM • VM can have multiple disk drives
Log File	• Keeps a log of VM activity • Is useful for troubleshooting
Virtual Configuration File	• Stores the configuration information chosen during VM creation • Includes information such as number of CPUs, memory, number and type of network adaptors, and disk types

Source: **EMC²**, Cloud Infrastructure and Service, Student Guide, 2011

I file della Virtual Machine (VM) 2/2

Location: **storage/datastore1/vmname**

File	Usage	Description
.vmx	*vmname*.vmx	Virtual machine configuration file
.vmxf	*vmname*.vmxf	Additional virtual machine configuration files
.vmdk	*vmname*.vmdk	Virtual disk characteristics
-flat.vmdk	*vmname*-flat.vmdk	Virtual machine data disk
.nvram	*vmname*.nvram or nvram	Virtual machine BIOS or EFI configuration
.vmem	*vmname*.vmem	Virtual machine paging backup file
.vmsd	*vmname*.vmsd	Virtual machine snapshots information (metadata) file
.vmsn	*vmname*.vmsn	Virtual machine memory snapshot file
.vswp	*vmname*.vswp	Virtual machine swap file
.vmss	*vmname*.vmss	Virtual machine suspend file
.log	vmware.log	Current virtual machine log file
-#.log	vmware-#.log (where # is a number starting with 1)	Old virtual machine log files
.scoreboard	vmware-#.scoreboard (where # is a number starting with 1)	Scoreboard files contain performance information about the VM

Source: **EMC²**, Cloud Infrastructure and Service, Student Guide, 2011

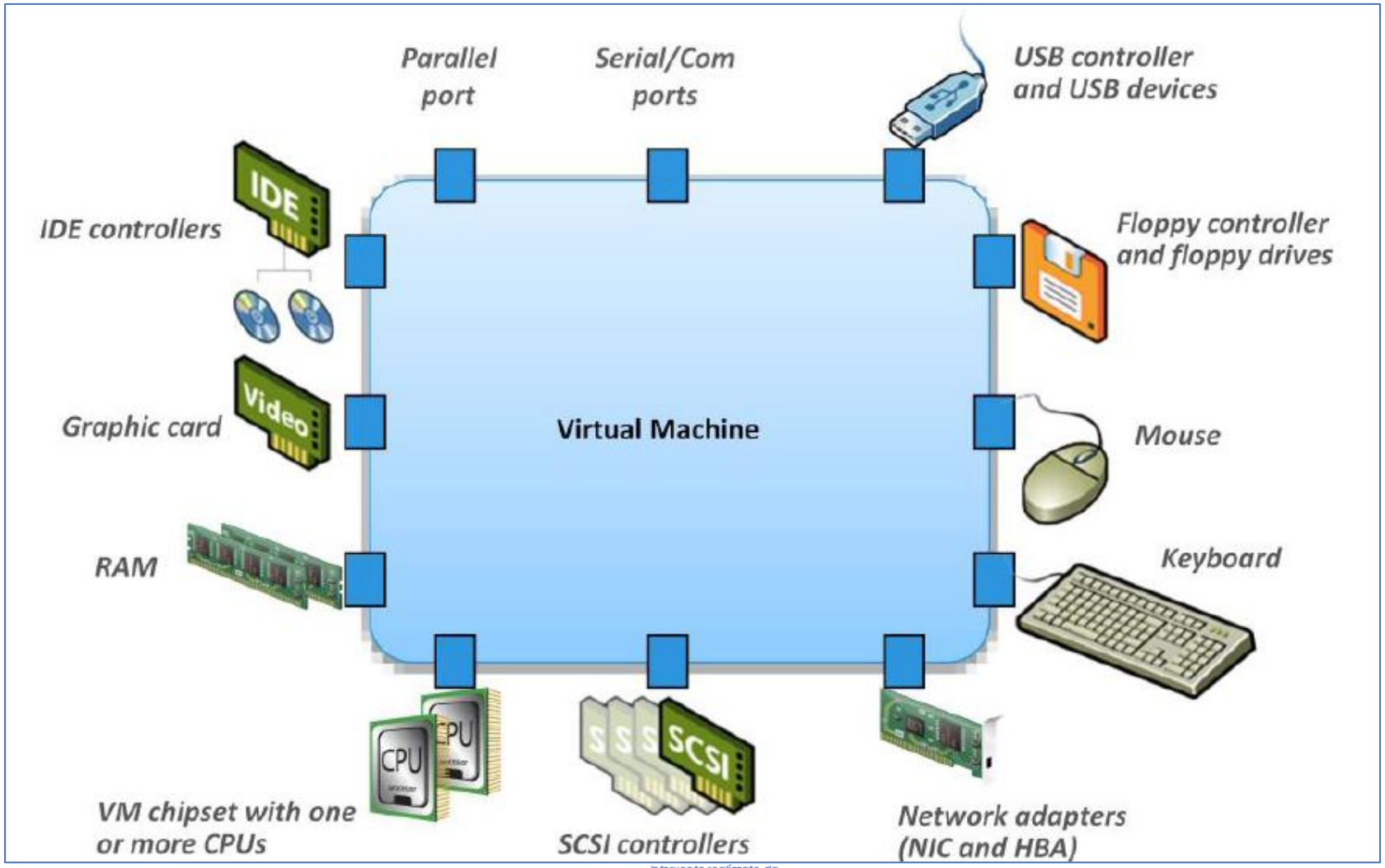
I file system per la gestione dei file di VM

I file system (formati) supportati da hypervisor sono:

- Virtual Machine File System (VMFS).
  - È un file aggregato che permette alle diverse macchine fisiche di leggere/scrivere sullo stesso storage contemporaneamente.
  - Può essere istanziato su storage esterno FC e iSCSI, oltre che su storage locale.
- Network File System (NFS).
  - Abilita l'archiviazione dei file di VM su un file server remoto (Network Attached Storage NAS).
  - Il client NFS è implementato in hypervisor.

Source: **EMC²**, Cloud Infrastructure and Service, Student Guide, 2011

L'HW della Virtual Machine



Source: **EMC²**, Cloud Infrastructure and Service, Student Guide, 2011

I componenti HW della Virtual Machine (VM)

Virtual Hardware	Description
vCPU	• Virtual machine (VM) can be configured with one or more virtual CPUs • Number of CPUs allocated to a VM can be changed
vRAM	• Amount of memory presented to the guest operating system (OS) • Memory size can be changed based on requirement
Virtual Disk	• Stores VM's OS and application data • A VM should have at least one virtual disk
vNIC	• Enables a VM to connect to other physical and virtual machines
Virtual DVD/CD-ROM Drive	• It maps a VM's DVD/CD-ROM drive to either a physical drive or an .iso file
Virtual Floppy Drive	• It maps a VM's floppy drive to either a physical drive or an .flp file
Virtual SCSI Controller	• VM uses virtual SCSI controller to access virtual disk
Virtual USB Controller	• Maps VM's USB controller to the physical USB controller

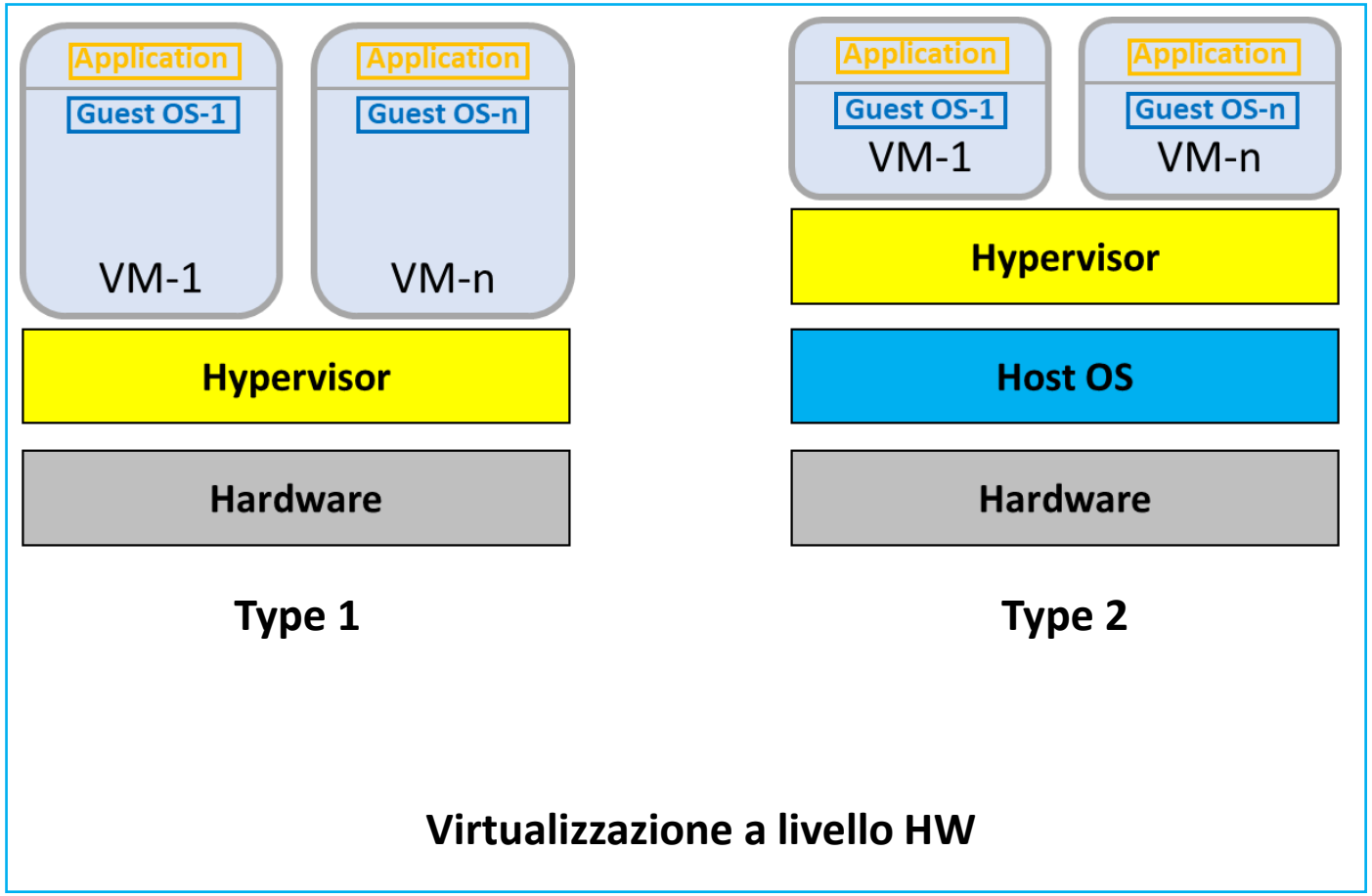
Source: **EMC²**, Cloud Infrastructure and Service, Student Guide, 2011

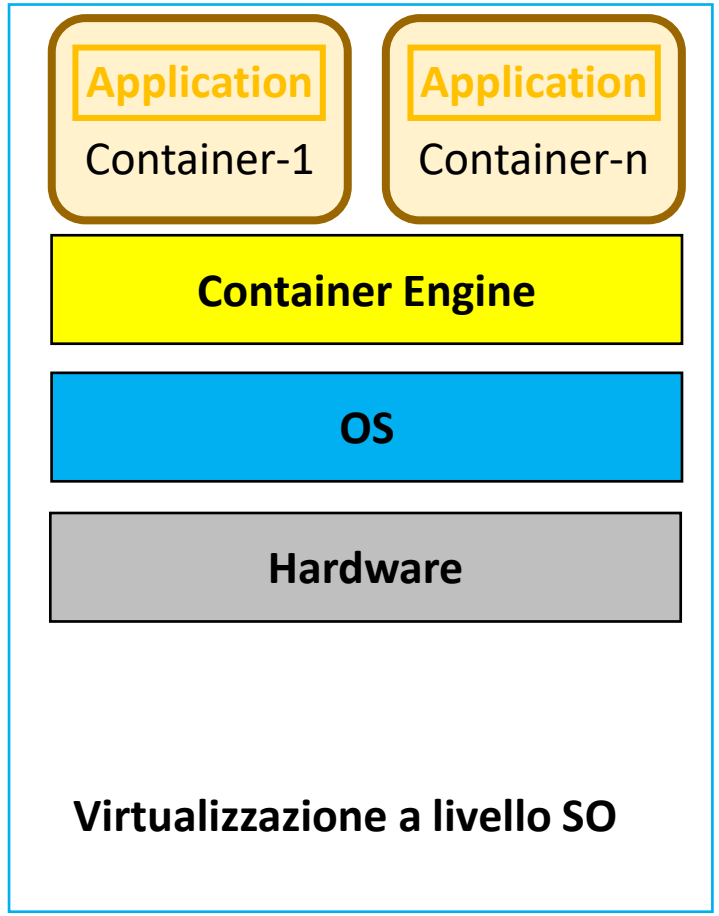
Container – Virtualizzazione a livello SO

La Virtualizzazione a livello di SO 1/2

- Finora abbiamo considerato la virtualizzazione a livello hardware.
- Analizziamo ora la virtualizzazione a livello di sistema operativo (o container-based virtualization).
- Permette di eseguire molteplici ambienti di esecuzione tra di loro isolati all'interno di un singolo SO.
  - Tali ambienti sono chiamati:
    - **container**;
    - jail;
    - zone;
    - virtual execution environment (VE).

La Virtualizzazione a livello di SO 2/2





Heavy-weight

Light-weight

La Virtualizzazione a livello di SO: Vantaggi

Rispetto alla virtualizzazione a livello HW.

- Minimo degrado delle prestazioni.
  - Le applicazioni invocano direttamente le system call, senza mediazioni di hypervisor.
- Tempo di startup e shutdown minimi.
  - Secondi per container, minuti per VM.
- Alta densità.
  - Centinaia di istanze sulla singola macchina fisica (PM).
- Dimensione dell'immagine più piccola (footprint).
  - Non include il kernel di SO.
- Possibilità di condividere le pagine di memoria tra i diversi container in esecuzione sulla stessa PM.
- Portabilità e interoperabilità migliori.
- Le applicazioni su container sono indipendenti dall'ambiente di esecuzione.

La Virtualizzazione a livello di SO: Svantaggi

Rispetto alla virtualizzazione a livello HW.

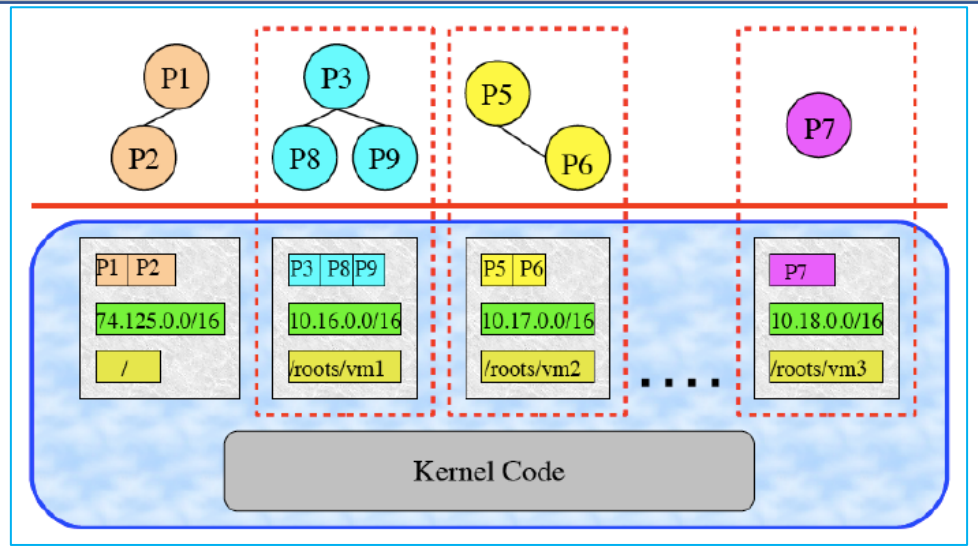
- Minore flessibilità.
  - Non posso coesistere diversi SO sulla stessa PM.
- Solo applicazioni native per i SO supportati (p. es. solo applicazioni native per Linux).
- Meno isolamento.
- Maggiore rischio di vulnerabilità.
  - La singola vulnerabilità nel kernel di SO compromette l'intero sistema.

Confronto tra Virtualizzazione a livello HW vs SO

	VMs	Containers
Memory	High Occupation	Low Occupation
Startup Time	Minutes	Seconds/Milliseconds
Performance	Limited	Native
Portability	Medium	High
Isolation	High	Medium
Efficiency	Low	High

Metodologie per Virtualizzazione a livello di SO (1)

- Il kernel del SO consente l'esistenza di diverse istanze isolate di spazi d'utente, chiamate container, che condividono però le risorse (di CPU, memoria e storage) di un SO host.
- Ogni container è costituito da:
  - **insieme di processi;**
  - **file system;**
  - **utenti;**
  - **interface di rete con relativi indirizzi IP;**
  - **tabelle di routing;**
  - **regole di configurazione del firewall, … .**
- I container condividono lo stesso SO, in particolare, il kernel di SO.



Solo Linux?

- Windows e OS X supportano la virtualizzazione basata su container.
  - Docker Desktop è una applicazione per Windows e Mac (o Linux) che permette di installare un Docker Engine su Windows OS > 10 o MacOS.
- Alternative: installare una VM con Linux come guest OS e utilizzare la virtualizzazione basata su SO all'interno della VM.
  - Ovviamente, impatto sulle prestazioni.
  - Le applicazioni containerizzate devono girare su Linux (non sono possibili applicazioni native OS X o Windows).

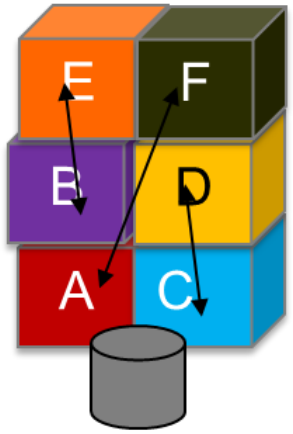
I container abilitano nuove tecnologie

- In sintesi, l'uso dei container permette di avere l'applicazione e tutte le sue dependencies (tutto il sw che serve per farla funzionare correttamente, in pratica i file bins/libs – binary e le librerie) in un singolo "contenitore", che permette quindi di eseguirlo pressoché in ogni ambiente.
- I container risultano più efficienti rispetto alle VM, perché utilizzano meno risorse.

Inoltre,

- I container sono la tecnologia chiave per abilitare i micro-servizi (microservice) e il cloud serverless (serverless computing).

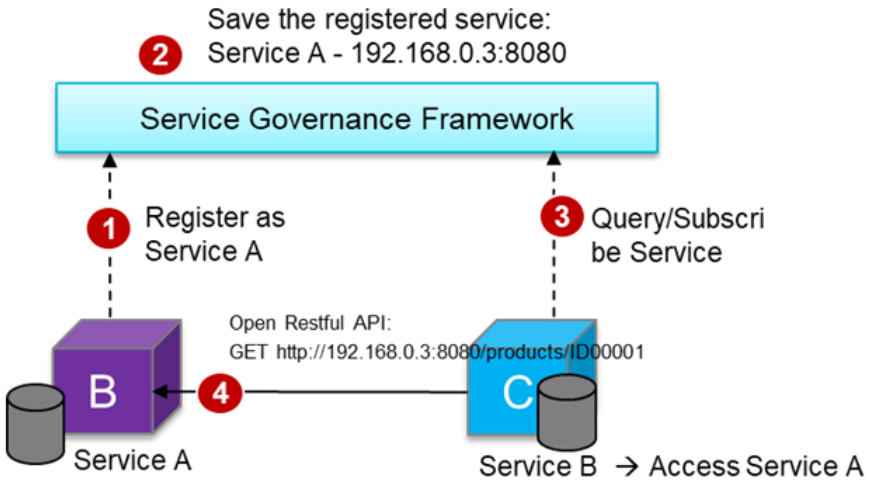
I Microservice



Monolithic System

All modules closely coupled

- **Internal Comm: function call**
- **Data: shared structure**
- **Dev: Top-Down, whole system**
- **Ops: Whole system**



Micro-service based System

Decentralized system – self registered/published services cooperate to provide functions – **Micro-services run on containers**

- **Internal Comm: dynamically discovered, open-API**
- **Data: independent from each other**
- **Dev: independent framework, language. Development in parallel**
- **Ops: each micro-service is self-contained, independently deployable, replaceable and upgradable, which brings agility to software**

Container as a Service – CaaS

- Anche la tecnologia di virtualizzazione a livello di SO, basata su container, viene usata dai provider per offrire i servizi del Cloud. Si parla in questo caso di **Container as a Service**.
- Il servizio cloud di CaaS permette di sviluppare e gestire delle applicazioni usando i container.
- Il CaaS si colloca tra IaaS e PaaS. Tuttavia sarebbe più appropriato classificarlo come un sottocaso di IaaS, in cui la risorsa base è rappresentata dal container.
- Tecnologia ampiamente diffusa sul mercato:
  - Amazon Elastic Container Service (ECS);
  - Microsoft Azure Container;
  - Google Container Engine;
  - Alauda (CaaS solution);
  - Docker Cloud.

Container Orchestration

- Quando un'applicazione è costituita da più container (*multi-container application*), distribuiti in cluster dinamici su larga scala, si rende necessario l'utilizzo di una piattaforma funzionale che gestisca il ciclo di vita dei suoi container costituenti.
- Questa piattaforma prende il nome di Orchestrator.
- L'orchestrator permette di configurare, attivare/istanziare (deployment), monitorare e controllare dinamicamente le applicazioni su container. Di fatto, integra e gestisce i container allocando le risorse rese necessarie dalle richieste degli utenti.
- Alcuni esempi:
  - **Kubernetes** (K8s);
  - Docker Swarm;
  - Amazon Elastic Container Service;
  - Google Kubernetes Engine;
  - Marathon;
  - Nomad (container orchestrator per Mesos).

} Servizi Cloud "fully-managed"

Il modello Google: tutto è su container

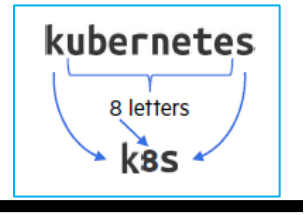
- Da anni Google ha cambiato il suo paradigma di gestione dei DC, da V-DC *machine-oriented* a V-DC *application-oriented*.
- Questo application-oriented shift, come viene chiamato, è stato possibile grazie alla trasformazione dei V-DC utilizzando la tecnologia dei container.

- *"Goal: let container technology operate at Google scale*.
  - *Everything at Google runs in containers*.
  - *Google launches several billion containers per week into Google Cloud*".

- Fondamentale il ruolo di un orchestrator nell'ottenere tali strabilianti prestazioni.
  - Borg and Omega, sono sistemi proprietari puramente interni a Google, predecessori di Kubernetes.
  - Kubernetes: piattaforma di container orchestration open-source.

Kubernetes (K8s)





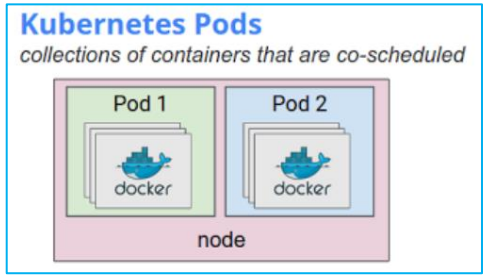
- "Google's Kubernetes, also known as K8s, is an open-source system for automating deployment, scaling, and management of containerized applications".
- http://kubernetes.io
- Principali caratteristiche:
  - *Portable: public, private, hybrid, multi-cloud;*
  - *Extensible: modular, pluggable, hookable, composable;*
  - *Self-healing: auto-placement, auto-restart, auto-replication, auto-scaling of containers.*
- Funziona su piattaforme di cloud pubblico e privato (AWS, Microsoft Azure, OpenStack, Apache Mesos) e anche su bare metal.
- Standard di fatto, offerto dalla maggioranza dei provider come servizio Cloud.

Source: http://kubernetes.io

Kubernetes (K8s): Pod

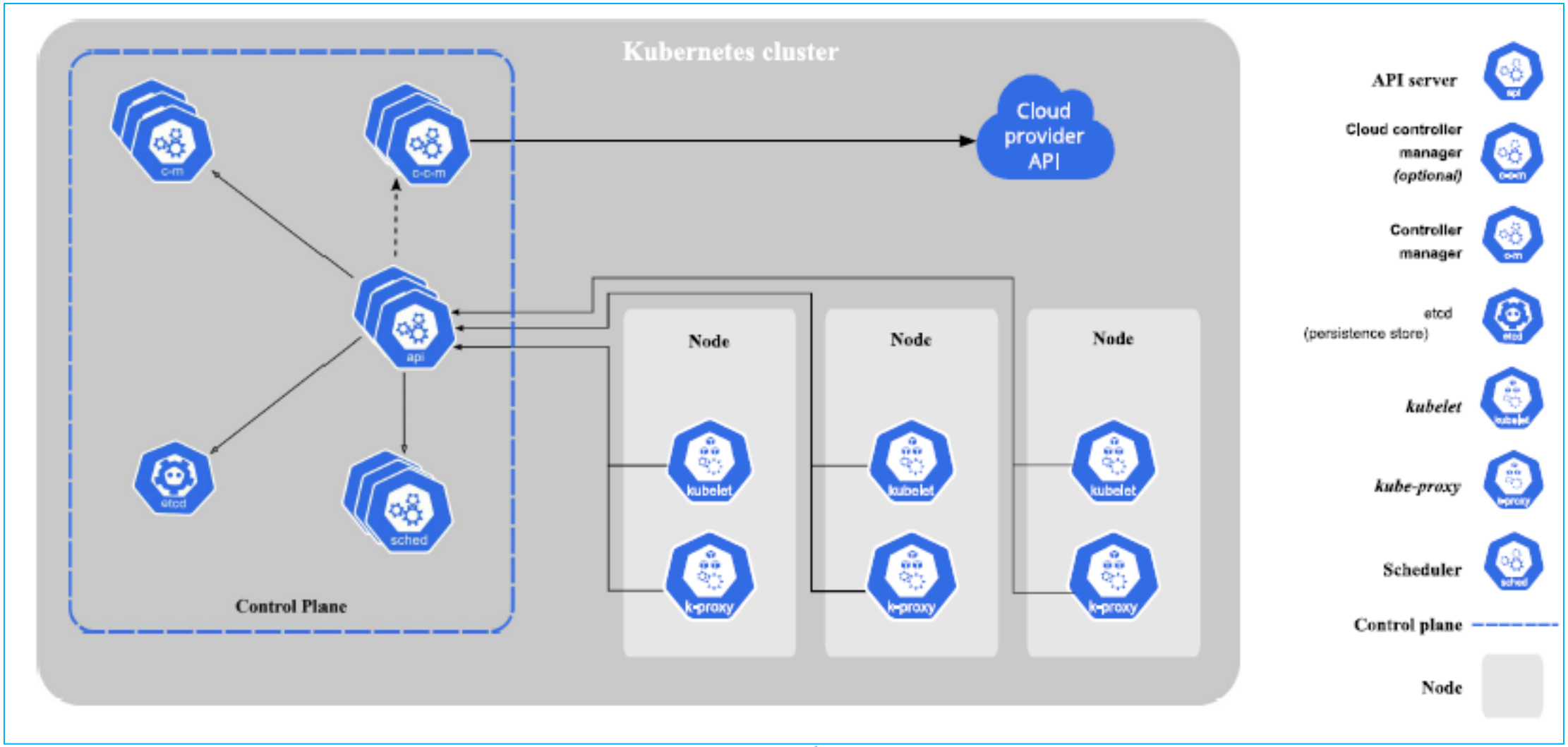


- Pod: the smallest deployable units of computing that you can create and manage in Kubernetes.
- A Pod (*as in a pod of whales or pea pod*) is a group of one or more (tightly coupled) containers, with shared storage and network resources, and a specification for how to run the containers.
- A Pod's contents are always co-located and co-scheduled, and run in a shared context.
- The shared context of a Pod is a set of Linux namespaces, cgroups, and potentially other facets of isolation - the same things that isolate a container. So a Pod is similar to a set of containers with shared namespaces and shared filesystem volumes.
- Kubernetes gives pods their own IP addresses and a single DNS name for a set of pods, and can load-balance across them.
- Users organize pods using labels:
  - Label: arbitrary key/value pair attached to pod;
  - E.g., role=frontend and stage=production.



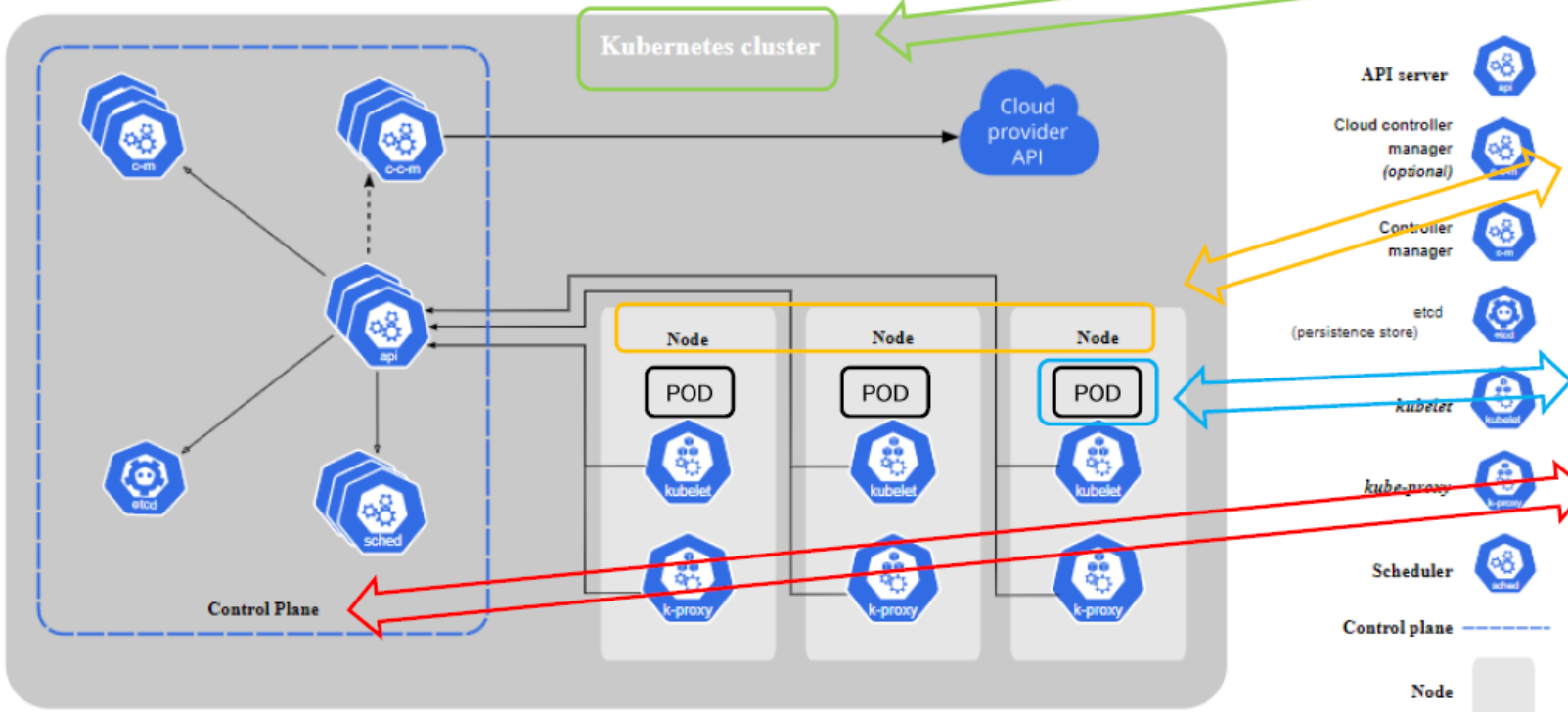
Source: http://kubernetes.io

Kubernetes (K8s): architettura 1/3



Source: https://kubernetes.io/docs/concepts/overview/components/

Kubernetes (K8s): architettura 2/3



When you deploy Kubernetes, you get a cluster.

A Kubernetes cluster consists of a set of worker machines, called nodes, that run containerized applications. Every cluster has at least one worker node.

The worker node(s) host the Pods that are the components of the application workload.

The control plane manages the worker nodes and the Pods in the cluster.

API server
Cloud controller manager *(optional)*
Controller manager
etcd (persistence store)
kubelet
kube-proxy
Scheduler
Control plane
Node

In production environments, the control plane usually runs across multiple computers and a cluster usually runs multiple nodes, providing fault-tolerance and high availability.

Source: https://kubernetes.io/docs/concepts/overview/components/

Kubernetes (K8s): architettura 3/3

- L'architettura di K8s è organizzata secondo un framework master-worker.

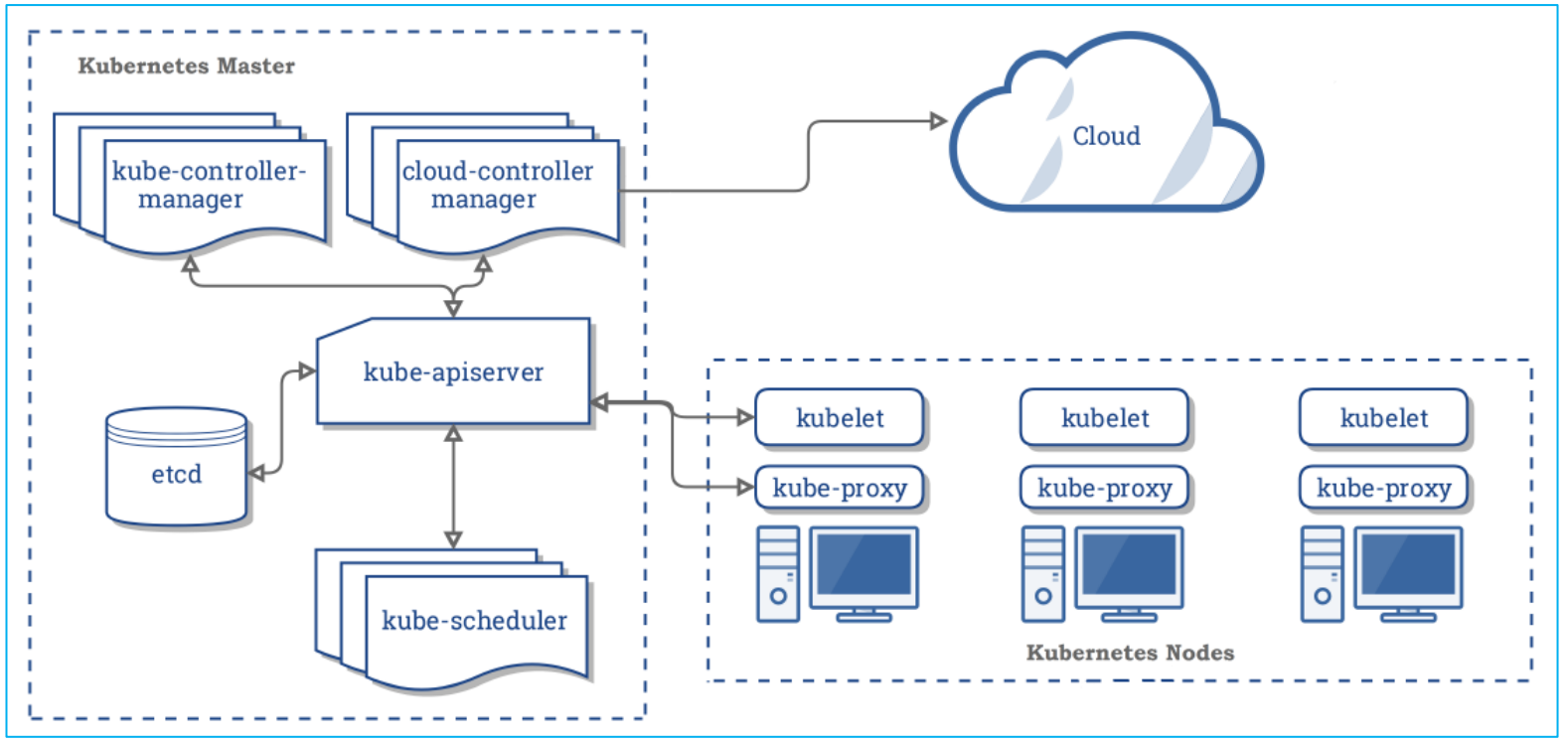
kube-apiserver
- exposes Kubernetes API

etcd
- key-value store
- backing store for cluster data

kube-scheduler
- Selects a node to run a POD
- Policy based scheduler

kube-controller-manager
- Node controller
- Endpoints controllers
- Service Account & Token controllers

cloud-controller-manager (optional)
- Link cluster with cloud provider's API



kubelet
- Agent that makes sure Pods are running and healthy

kube-proxy
- Network proxy
- Maintains network rules for connecting to Pod
- Manages internal and external cluster communications

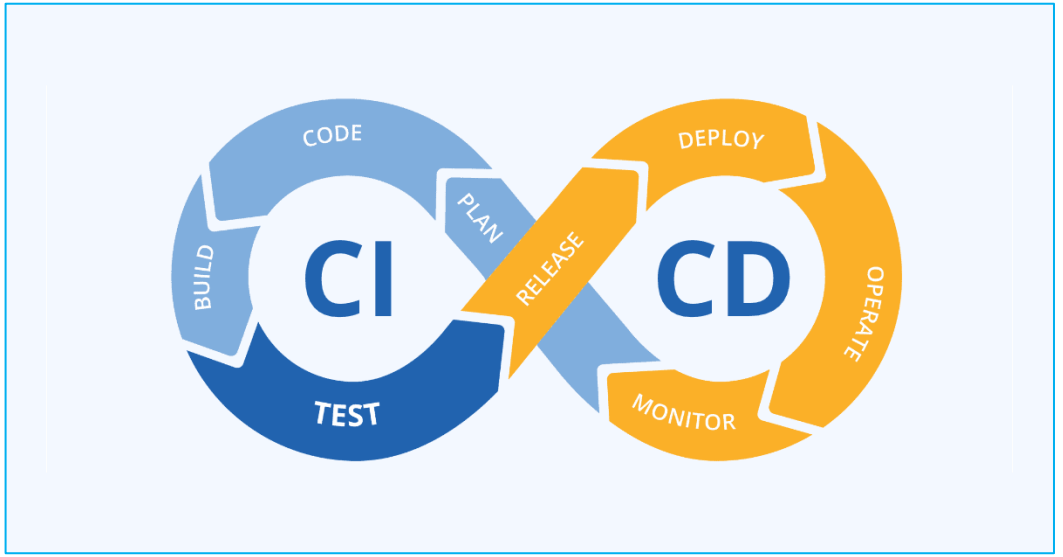
container runtime

docker containerd cri-o

Un approccio innovativo allo sviluppo del sw 1/3

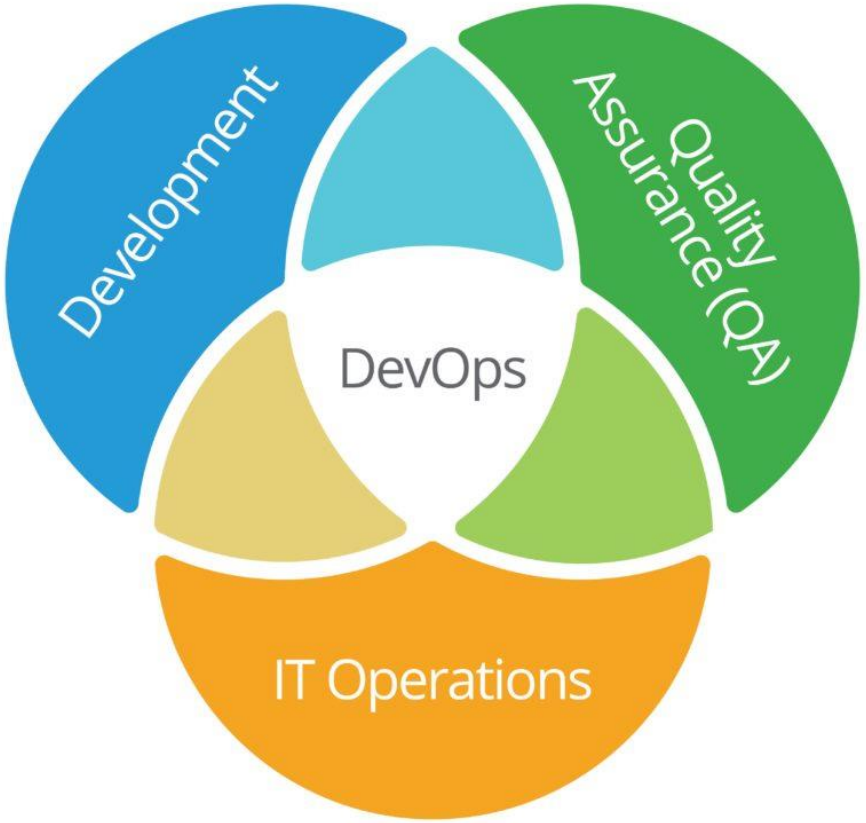
- I container forniscono una nuovo modo per costruire, impacchettare, condividere e distribuire le applicazioni.
- In particolare, i container supportano l'introduzione di nuove metodologie per lo sviluppo e la distribuzione del sw: CI/CD e DevOps e Agile.

- **Continuous Integration and Continuous Deployment CI/CD**: questo paradigma descrive le fasi chiave in un flusso automatico di sviluppo e distribuzione del sw. Questo flusso include *design, coding, testing, integrazione, delivery, validation e deployment*, prima della fase di operation nell'ambiente finale.
- I container (più della VM) permettono agli sviluppatori di creare codice in modo collaborativo, condividendo facilmente le immagini sw, mentre semplificano la distribuzione verso le differenti infrastrutture.



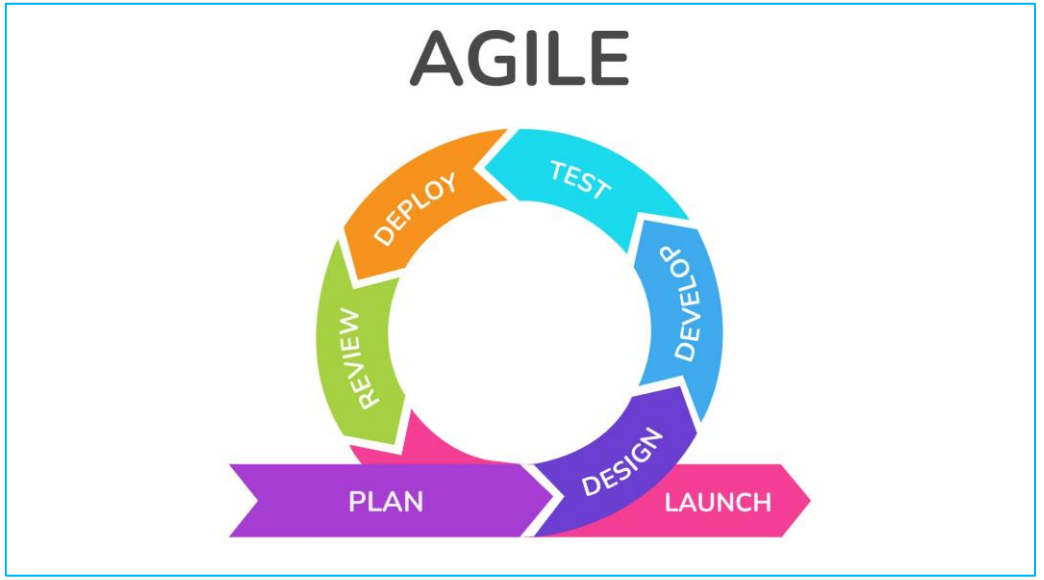
Un approccio innovativo allo sviluppo del sw 2/3

- **Development and Operations DevOps**: Metodologia di sviluppo sw creata per ridurre o eliminare il divario tra sviluppo e operation, enfatizzando comunicazione, collaborazione, integrazione, quality assurance e delivery con un deployment automatico.
- Di fatto, la metodologia vuole bilanciare l'aspirazione per veloci cambiamenti del team di sviluppo con il desiderio di stabilità del team di operation, attraverso comunicazione e collaborazione efficaci.



Un approccio innovativo allo sviluppo del sw 3/3

- DevOps è una pratica di sviluppo agile e una mentalità che utilizza principi agili (collaborazione, comunicazione, utilizzo degli strumenti giusti) per semplificare la costruzione, il test e il rilascio del software.
- CI/CD è una tattica DevOps, che utilizza i giusti strumenti di test automatizzati per implementare lo sviluppo agile.
- **Agile** è un ampio framework che comprende le altre due pratiche. Si concentra sui processi che evidenziano il cambiamento mentre accelera la distrubuzione. È una filosofia di sviluppo software che viene implementata in varie metodologie: Scrum, Kanban, Scaled Agile Framework (SAFe), ecc.
- Tutte e tre le pratiche hanno lo stesso obiettivo: **creare software migliore in minor tempo**. Queste pratiche si sovrappongono quasi sempre.



IaC

Infrastructure as Code (IaC)

- **Infrastructure as Code** (IaC) è un approccio che permette di definire, creare e gestire l'infrastruttura IT tramite **codice**, invece di configurazioni manuali.
- Concetti chiave:
  - ✓ Infrastruttura descritta con file di testo
  - ✓ Versionabile (Git)
  - ✓ Automatizzabile e ripetibile (*idempotenza*)
  - ✓ Specifico lo **stato desiderato** (IaC *dichiarativa*, la più recente [2010] vs IaC *procedurale/imperativa*)
- Quale infrastruttura?
  - ➢ Container Docker
  - ➢ Volumi e reti
  - ➢ VM e cluster Kubernetes (anche su AWS o Azure)
- Esempio: creo VM, rete e storage scrivendo codice invece di usare una console cloud

Infrastructure as Code (IaC): Perchè?

- IaC risolve i limiti della gestione manuale
  - ✓Coerenza tra ambienti (dev / test / prod)
  - ✓Riproducibilità dell'infrastruttura
  - ✓Riduzione degli errori umani
  - ✓Velocità di creazione degli ambienti

- Il **configuration drift** è la situazione in cui lo stato reale dell'infrastruttura non coincide più con lo stato definito nel codice (IaC).
  - ➢In pratica: il codice dice una cosa, l'infrastruttura reale ne fa un'altra
    - ✓Qualcuno modifica a mano un server (SSH, console cloud)
    - ✓Una configurazione cambia fuori dal codice
    - ✓Script procedurali vengono eseguiti solo parzialmente

Infrastructure as Code (IaC)

Come si usa (flusso tipico)?

- Scrivo codice IaC
- Versiono su Git
- *plan* → anteprima
- *apply* → crea/modifica infrastruttura

Infrastructure as Code (IaC): dichiarativa vs procedurale

1. Crea una VM
2. Installa Docker
3. Avvia il container
4. Crea directory

Descrive i passi da eseguire, nell'ordine, per ottenere l'infrastruttura desiderata

Voglio:
- 1 VM
- 1 container in esecuzione
- 1 volume persistente

Descrive lo stato finale desiderato dell'infrastruttura, senza specificare i passi per arrivarci

Infrastructure as Code (IaC): Come si implementa?

- File di definizione (codice)
- Motore che applica lo stato desiderato

Dipendenze tecnologiche: IaC non vive da sola, dipende da:

  - Virtualizzazione (VM, hypervisor)
  - Container (Docker, Kubernetes)
  - Cloud (pubblico, privato, ibrido)
  - API dell'infrastruttura

- IaC non è solo cloud: vale anche per data center on-prem
- Use case: Creare ambienti test e produzione identici

Infrastructure as Code (IaC): Architettura

[File IaC]

↓

[Motore IaC]

↓

[Provider / Driver]

↓

[Infrastruttura reale]

Infrastructure as Code (IaC): Come si implementa?

- Il motore è il componente che interpreta il codice IaC e lo trasforma in azioni reali sull'infrastruttura, chiamando le API del cloud o della piattaforma sottostante
- Cosa fa concretamente un motore IaC?
  - Legge il codice/template (YAML, JSON, HCL, Bicep, ecc.)
  - Valida che sia corretto
  - Calcola lo stato desiderato
  - Confronta lo stato desiderato con quello reale
  - Decide le operazioni da eseguire (create / update / delete)
  - Chiama le API dell'infrastruttura
  - Tiene traccia dello stato (implicitamente o esplicitamente)

Infrastructure as Code (IaC): Linguaggi e Strumenti

Strumento	Linguaggio	Produttore	Open Source	Tipo	Uso tipico
Terraform	HCL	HashiCorp	Parziale	Dichiarativo	Multi-cloud
OpenTofu	HCL	Linux Foundation	Sì	Dichiarativo	Fork di Terraform per restare open-source
Ansible	YAML	Red Hat	Sì	Procedurale	Configurazione OS/app
CloudFormation	YAML/JSON	AWS	No	Dichiarativo	Solo AWS
ARM	Bicep/ARM JSON	Microsoft Azure	Si	Dichiarativo	Solo Azure

Infrastructure as Code (IaC): Vantaggi

- **Source of Truth**: il codice è la verità ufficiale
- Ambienti coerenti e ripetibili
- Automazione e velocità
- Base del DevOps
- Integrazione con GitHub
  - ✓Codice IaC in repository Git
  - ✓Pull Request = modifica infrastruttura
  - ✓CI/CD → deploy automatico dell'infra

❑ATT: errore di codice = errore di infra!

Infrastructure as Code (IaC): alcuni provider ufficiali Terraform

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

Infrastructure as Code (IaC): alcuni provider ufficiali Terraform

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "eu-west-1" # esempio: Irlanda
}

##############################################
# 2. Region corrente (lettura dinamica)
##############################################
data "aws_region" "current" {}
```

Infrastructure as Code (IaC): alcuni provider ufficiali Terraform

```
##############################################
# 3. Availability Zones
# Le AZ sono unità di isolamento fisico
# Concettualmente ≃ datacenter
##############################################
data "aws_availability_zones" "available" {
  state = "available"
}

##############################################
# 4. VPC esistente
# In AWS la VPC è il perimetro di rete principale
##############################################
data "aws_vpc" "selected" {
  tags = {
    Name = "main-vpc"
  }
}
```

Infrastructure as Code (IaC): alcuni provider ufficiali Terraform

```
##############################################
# 5. Subnet esistente
# Le subnet sono sempre:
# - dentro una VPC
# - associate a UNA sola AZ
##############################################
data "aws_subnet" "public_a" {
  vpc_id            = data.aws_vpc.selected.id
  availability_zone = data.aws_availability_zones.available.names[0]

  tags = {
    Name = "public-a"
  }
}
```

Infrastructure as Code (IaC): alcuni provider ufficiali Terraform

```
##############################################
# 6. Security Group
# Firewall STATEFUL
# Associato a ENI / EC2 / LB
##############################################
data "aws_security_group" "web_sg" {
 name   = "web-sg"
 vpc_id = data.aws_vpc.selected.id
}

##############################################
# 7. Network ACL (NACL)
# Firewall STATELESS
# Associato alle subnet
##############################################
data "aws_network_acl" "main" {
 vpc_id = data.aws_vpc.selected.id

 tags = {
  Name = "main-nacl"
 }
}
```

Infrastructure as Code (IaC): alcuni provider ufficiali Terraform

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0"
    }
  }
}
```

Infrastructure as Code (IaC): alcuni provider ufficiali Terraform

```
terraform {
  required_providers {
    google = {
      source  = "hashicorp/google"
      version = "~> 5.0"
    }
  }
}
```

Infrastructure as Code (IaC): alcuni provider ufficiali Terraform

```
terraform {
  required_providers {
    kubernetes = {
      source  = "hashicorp/kubernetes"
      version = "~> 2.0"
    }
  }
}
```

Infrastructure as Code (IaC): alcuni provider ufficiali Terraform

```
terraform {
  required_providers {
    vsphere = {
      source  = "hashicorp/vsphere"
      version = "~> 2.0"
    }
  }
}
```